Guia paso a paso: crear un juego de plataformas con GlasnostFWK

Esta guia describe el proceso completo para disenar e implementar un juego de plataformas sobre el framework GlasnostFWK (Z80, Amstrad CPC, CPCTelera Next). Se asume familiaridad basica con ensamblador Z80 y con el manual del framework.

1. Concepto y alcance

1. Define el objetivo del jugador: recoger objetos, llegar a la salida, derrotar enemigos.
2. Define el numero de niveles, duracion objetivo y dificultad.
3. Elige el perfil de video: estandar 160x200 (recomendado para 50 Hz) o experimental 160x288.
4. Decide si usas tiles de 8x8 o 16x16 y el tamano del viewport.

2. Arquitectura del proyecto

Crea un directorio de ejemplo con la estructura habitual:

```
examples/miplataforma/
  example.json
  main.s
  assets.s
```

En `example.json` declara los modulos necesarios:

```
{
  "name": "miplataforma",
  "profile": "standard-160x200.json",
  "modules": [
    "core/system.s",
    "core/input.s",
    "core/pool.s",
    "video/video.s",
    "extensions/tilemap.s",
    "extensions/tilemap_render.s",
```

```
    "extensions/physics_8_8.s",
    "extensions/motion_u8.s",
    "extensions/platform_tilemap.s",
    "extensions/animation.s",
    "ui/text.s",
    "ui/window.s"
  ],
  "game_sources": ["main.s", "assets.s"]
}
```

3. Mapa y tiles

1. Crea un mapa de tiles como array de bytes (ancho x alto).
2. Define la tabla de tipos de colision: solido, plataforma de una direccion, vacio.
3. Crea el tileset grafico (bloques de N bytes por tile).
4. Usa `FWK_TILEMAP_DESCRIPTOR` y `FWK_TILE_RENDER_DESCRIPTOR`.
5. Renderiza con `fwk_tilemap_render_view` y actualiza celdas con `fwk_tilemap_render_cell`.

> Usa actualizaciones parciales (dirty rectangles) para evitar redibujar todo el mapa cada frame.

4. Jugador y entidades

1. Define un estado de jugador: x, y, vx, vy, en suelo, animacion.
2. Usa un pool (`fwk_pool`) para enemigos y objetos.
3. Implementa entrada con `fwk_input`: izquierda, derecha, salto.
4. Aplica gravedad y movimiento con `physics_8_8` o `motion_u8`.

5. Colisiones

1. Colision con el mapa mediante `fwk_tilemap_query` o `platform_tilemap`.
2. Resuelve ejes por separado: primero X, luego Y.
3. Colision entre cuerpos con `fwk_collision_aabb` para enemigos y proyectiles.
4. Marca el jugador como "en suelo" cuando la colision inferior es solida.

6. Salto y coyote time

1. Permite saltar solo si el jugador esta en suelo o dentro de una ventana de coyote time.
2. Aplica impulso vertical negativo al saltar.
3. Variable jump height: si se suelta el boton antes del apex, reduce la velocidad vertical.

7. Camara

1. Sigue al jugador con un offset (centrado o tercio superior).
2. Limita la camara a los bordes del mapa.
3. Actualiza la camara antes de renderizar el viewport.

8. Sprites y animacion

1. Define descriptores de sprite con `FWK_SPRITE_DESCRIPTOR`.
2. Usa `fwk_sprite_draw` para sprites opacos y `fwk_sprite_draw_masked` para transparencia.
3. Gestiona animaciones con `fwk_animation`: cursor de frames y cambio de descriptor.
4. Ordena sprites solapados con `fwk_sprite_batch` si es necesario.

9. Enemigos y objetos

1. Define tipos de enemigo: patrulla, volador, estatico.
2. Logica de IA simple: cambiar direccion al chocar con pared o borde.
3. Objetos coleccionables: monedas, vidas, power-ups.
4. Elimina entidades inactivas del pool.

10. HUD y ventanas

1. Muestra vidas, puntuacion y nivel con `fwk_text`.
2. Usa `fwk_window` para menus de pausa y game over.
3. Captura y restaura el fondo al abrir/cerrar ventanas.

11. Audio

1. Integra musica con `fwk_audio` (AKG).

2. Dispara SFX para salto, moneda, dano y muerte.
3. Controla volumen y parada con comandos.

12. Estados del juego

1. Estados: titulo, jugando, pausa, game over, victoria.
2. Transiciones claras y reinicio de nivel.
3. Guardado de mejor puntuacion en RAM o disco.

13. Optimizacion y rendimiento

1. Mide el coste con el arnes `examples/perf`.
2. Evita limpiar toda la pantalla cada frame.
3. Usa actualizaciones parciales y sprites enmascarados solo cuando sea necesario.
4. Objetivo: mantener 50 Hz (79872 T-estados/frame).

14. Pruebas y validacion

1. Escribe pruebas host para fisica y colisiones.
2. Usa el smoke test en CPC 6128 y 464.
3. Verifica en hardware real si es posible.

15. Distribucion

1. Empaqueta con `make release VERSION=x.y.z`.
2. Publica en GitHub Releases.
3. Incluye instrucciones y checksums.

> Esta guia es un resumen; consulta el manual en `docs/manual.html` y los ejemplos del framework para detalles de API.